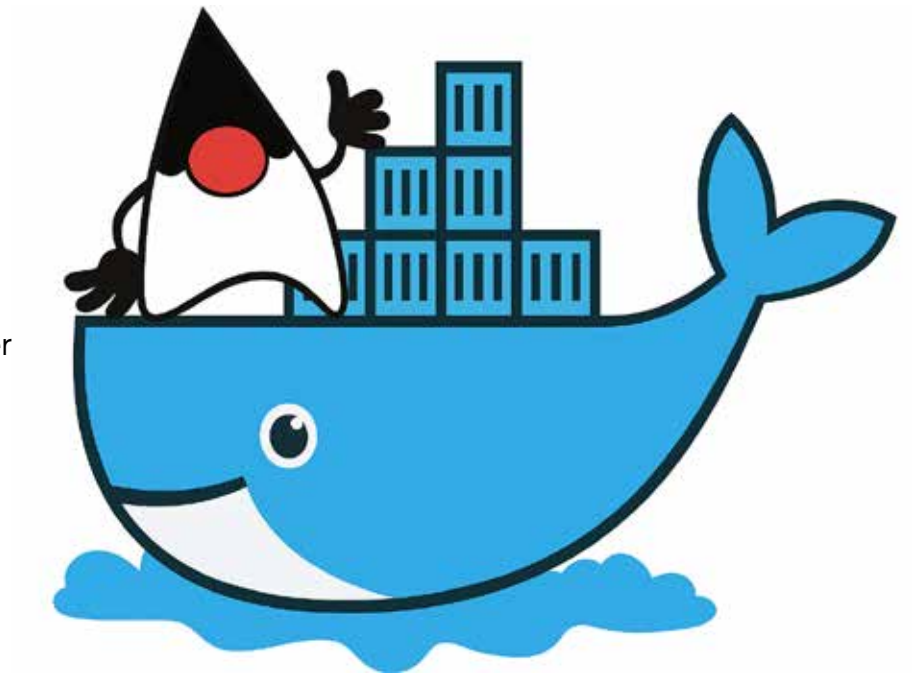
A Complete Guide to Deploying Java Applications in Docker

Explore the entire life cycle of deploying Java applications via Docker -- from understanding the basics and writing the initial Dockerfile to building container images and managing complex multi-container deployments using Docker Compose. You can use this guide to streamline the deployment pipeline of a Spring Boot microservice, a traditional Java SE program, and even an enterprise web application.



Consistency, scalability, and portability have become must-haves in software development. Because of Java's robustness, platform independence, and vast ecosystem, it has been the cornerstone of enterprise and backend applications. But rolling out Java applications on multiple environments -- develop, test, and production --can cause dependency hell among other issues.

Docker offers a new method of containerisation on the packaging and delivery end. It wraps a Java application with its dependencies in a thin container that is portable and can be run anywhere -- on a local machine, in some test environment, or on a cloud-hosted production server.

The problem that Docker solves

An all-too common and annoying problem for the software developer is: "It works on my machine." The Java application may be working flawlessly on the developer's system, say with JDK 17 on macOS, but in QA or production it may be misbehaving according to different platform OSs and Java versions. Such variations break builds, or improperly configure sources or runtime situations.

These could be environment-related bugs -- the worst kind to locate and fix. Teams spend hours establishing how an issue can be replicated; in fact, it's not really because of faulty code but an inconsistent setup. This slows development cycles, delaying releases, and increasing deployment risks.

Docker resolves all that by allowing developers to pack the Java application with the specific runtime, libraries, environment variables and configurations into the container image that simply runs the same way in any environment where Docker is installed --however inefficiently configured it may be in development, test, and production instruments.

With Docker there is no need for installing Java inside the working environment or even resolving dependencies. The container already goes with whatever the app needs for execution. This prevents environmental bugs, allowing faster onboarding of developers and testing. Simply put, Docker makes it certain that a Java application runs smoothly everywhere, thus taking care of reliability and deployment ease.

Why use Docker for Java applications?

Imagine developing a Java game on your computer which functions perfectly when you play it but fails on your friend's system. The reason for this failure lies in the differences between your computer and your friend's system, which can exhibit Java version discrepancies and missing files.